CheckIn Docker and CI/CD Report

Oct 28, 2026 · @lindel

Summary

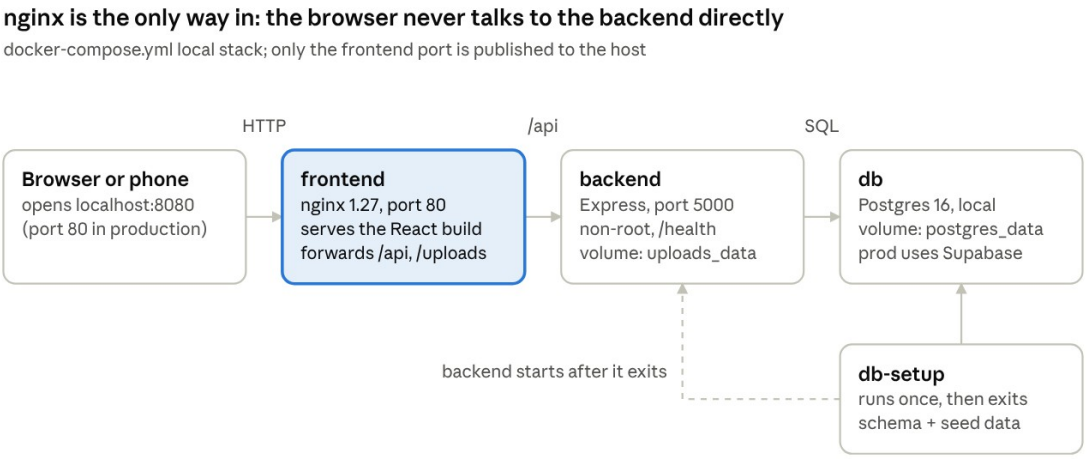
CheckIn now runs fully in Docker: 4 containers built from 2 Dockerfiles. The stack was running and healthy on 28 Oct 2026, and the project's smoke test passed all 5 checks. The 5-job GitHub Actions pipeline is written, but it has never run, because the `feature/ci-cd-docker` branch has not been pushed or merged into `dev`.

Area	What exists	State on 28 Oct 2026
Dockerfiles	`backend/Dockerfile` (3 stages, 2 targets), `frontend/Dockerfile` (2 stages)	Built locally
Images	`checkin-backend` 458 MB, `checkin-frontend` 77.2 MB, `db-setup` 540 MB	Built locally, not yet on GHCR
Local stack	`docker-compose.yml`: Postgres, db-setup, backend, frontend	Running, all healthy, smoke test passed
Production stack	`docker-compose.prod.yml`: GHCR images + Supabase	Written, not deployed
CI/CD pipeline	`.github/workflows/ci-cd.yml`, 5 jobs	Written, never run on GitHub
Cloud hosting	`render.yaml` (backend), `vercel.json` (frontend)	Written
Docs	`docs/DEPLOYMENT.md`, `docs/MOBILE.md`, README section	Written

All of this work was done by Damchey Lhendup in 3 commits between 12 and 28 Oct 2026 (see History).

Architecture

All traffic goes through the frontend's nginx container. The backend and database are reachable only inside Docker's private network.



Container architecture · 4 services

In production (`docker-compose.prod.yml`) the `db` and `db-setup` containers are dropped, and the backend connects to Supabase instead.

Dockerfiles and images

Both Dockerfiles are multi-stage builds. Build tools stay in the early stages, and only what the app needs at run time goes into the final image.

Backend: `backend/Dockerfile`

Stage	Built from	What it does
`base`	`node:22-alpine`	Adds OpenSSL, which Prisma needs on Alpine
`deps`	`base`	`npm ci` with dev dependencies, then `prisma generate`
`dbsetup` (target)	`deps`	One-off job image that runs `docker/db-setup.sh` to build a local database
`runtime` (default target)	`base`	`npm ci --omit=dev`, copies in the generated Prisma client, runs `node src/server.js` on port 5000

Security and reliability details in the runtime image:

- Runs as the non-root `node` user; only the `uploads/` folder is writable.
- Sets `NODE_ENV=production` and clears the npm cache to keep the image smaller.
- Has a health check that calls `GET /health` every 10 s, which Docker and compose use to decide when the backend is ready.
- `.dockerignore` keeps `.env` files, `node_modules`, uploads and the private timetable SQL (`timetable_AS2026.sql`) out of the image.

Frontend: `frontend/Dockerfile`

Stage	Built from	What it does
`builder`	`node:22-alpine`	`npm ci` and `npm run build` (Vite), with `VITE_API=/api`
final	`nginx:1.27-alpine`	Serves the built `dist/` files on port 80, with a health check

`frontend/nginx.conf` makes the frontend container the single entry point:

- `/api/` and `/uploads/` are forwarded to `backend:5000`, so the browser never needs the backend's address. One image therefore works on any host.
- Uploads up to 10 MB are allowed, matching the backend's multer limit.
- `/assets/` files (hashed by Vite) are cached for a year; `index.html` is never cached.
- Unknown paths fall back to `index.html`, so React Router links work on refresh.

Images on this laptop

Image	Size	Built
`checkin-backend:local`	458 MB	12 days ago
`checkin-frontend:local`	77.2 MB	11 days ago
`checkin-db-setup:latest`	540 MB	12 days ago

Containerisation: running the stack

There are two compose files: one for a laptop or CI, with its own database, and one for a production server that uses Supabase.

Local and CI: `docker-compose.yml`

Service	Image	Role	Starts when
`db`	`postgres:16-alpine`	Local PostgreSQL, data kept in the `postgres_data` volume	First
`db-setup`	backend, `dbsetup` target	Creates tables, loads the seed and sets test passwords, then exits	`db` is healthy
`backend`	backend, `runtime` target	Express API on port 5000 (not published to the host)	`db-setup` finished successfully
`frontend`	frontend image	nginx on host port `APP_PORT` (default 8080)	`backend` is healthy

How the local database is built (`backend/docker/db-setup.sh`):

1. Refuses to run if `DATABASE_URL` points at Supabase, so the shared database can't be overwritten by mistake.
2. If the tables already exist, it only reloads the seed and skips the remaining steps.
3. Otherwise it creates the `gen_id()` function and ID sequences (`db-bootstrap.sql`), then runs `prisma db push`.
4. It adds the Supabase-only `academic_year` column (`db-legacy.sql`), loads `prisma/seed.js`, and sets every test account's password to `Test@1234`.

To run it:

```
cp .env.example .env          # then set JWT_SECRET
docker compose up -d --build
# open http://localhost:8080 and log in as U001 / Test@1234
```

Checked on 28 Oct 2026: all 4 containers were up (db, backend and frontend healthy, db-setup exited with code 0), with the frontend on port 8081. `scripts/smoke-test.sh` passed all 5 checks:

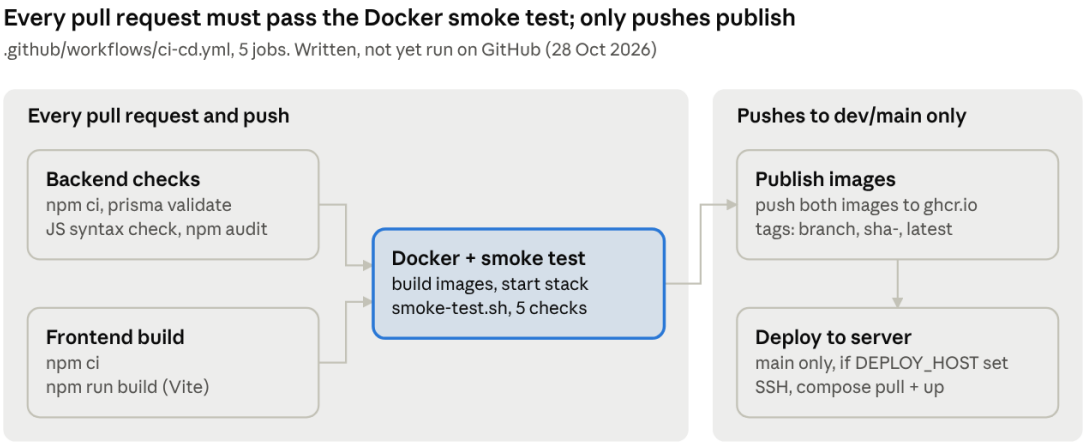
- ☒ Frontend serves the React app
- ☒ A deep link falls back to `index.html`
- ☒ Login as U001 through `/api` returns a token
- ☒ `GET /api/me` with the token returns U001
- ☒ `GET /api/me` without a token returns 401

Production: `docker-compose.prod.yml`

This file pulls the published images from GitHub Container Registry (`ghcr.io/ugaynobu/checkin-backend` and `checkin-frontend`) instead of building them. There is no database container; the backend reads the Supabase connection from `backend.env` on the server. The frontend is published on port 80, and uploads are kept in the `uploads_data` volume.

CI/CD pipeline

The GitHub Actions workflow tests every change in the same containers that production uses, and publishes images only after the smoke test passes.



CI/CD pipeline · 5 jobs, 2 trigger groups

Backend checks and Frontend build run side by side. If either fails, nothing else runs. Pull requests that are updated again cancel their older run.

Job	Runs on	Steps	Needs
Backend checks	Every PR and push	Node 22, `npm ci`, `prisma validate`, `prisma generate`, `node --check` on every JS file, `npm audit` (fails on critical issues only)	—
Frontend build	Every PR and push	Node 22, `npm ci`, `npm run build` with `VITE_API=/api`	—
Docker build + smoke test	Every PR and push	Makes a random `JWT_SECRET`, `docker compose build`, `up -d`, runs `scripts/smoke-test.sh`, prints logs if it fails, always runs `down -v`	Both checks
Publish images	Pushes to `dev`/`main` and `v*` tags	Logs in to ghcr.io with the workflow token, builds both images with Buildx and layer caching, tags them branch / `sha-xxxxxxx` / version / `latest` (main only)	Docker job
Deploy to server	Pushes to `main` when `DEPLOY_HOST` is set	Copies `docker-compose.prod.yml` over SSH, pulls the `sha-` tagged images, restarts with `up -d`, prunes old images	Publish

The deploy job uses the exact `sha-` tag of the commit, so a rollback means redeploying an earlier tag. It reads `DEPLOY_SSH_KEY` from secrets and `DEPLOY_USER`, `DEPLOY_PORT` and `DEPLOY_PATH` from repository variables.

Cloud hosting route

As a second option, added on 28 Oct 2026: `render.yaml` deploys the backend to Render's free plan (Singapore region, `GET /health` check, auto-deploy on push, `JWT_SECRET` generated by Render). `frontend/vercel.json` hosts the built frontend on Vercel, with page-link fallback and one-year caching for `/assets/`. This route doesn't use the Docker images.

History

Date	Commit	Change
28 Oct 2026	`7a2a883` dockerfiles	Added `render.yaml` and `vercel.json` for cloud hosting, plus phone install support (app icons, manifest, service worker) and `docs/MOBILE.md`
14 Oct 2026	`dc4860f` merge dev	Merged `dev` into the branch and updated Docker for the restructured backend (`src/server.js`)

Date	Commit	Change
12 Oct 2026	`bc86f8a` Add Docker	Both Dockerfiles, `nginx.conf`, both compose files, the db-setup scripts, the CI/CD workflow, the smoke test and `docs/DEPLOYMENT.md` (22 files, 765 lines)

Gaps and next steps

The Docker setup works locally. What's left is getting the pipeline running on GitHub and choosing one deployment route.

1. **Push the branch and open a pull request into `dev`.** `feature/ci-cd-docker` exists only on this laptop and is 3 commits ahead of `dev`. The workflow runs only on pull requests and pushes to `dev` or `main`, so it has never run.
2. **Run the unit tests in CI.** The workflow syntax-checks and builds the code but runs no tests. Add `npm test` to the Backend checks and Frontend build jobs, so the 41 unit tests run on every pull request. This is the testing manual's optional `test.yml` step.
3. **Choose one production route.** Two are set up: (a) GHCR images deployed to a server over SSH, which needs the `DEPLOY_HOST` variable, a `DEPLOY_SSH_KEY` secret and HTTPS for check-in; or (b) Render for the backend and Vercel for the frontend, which is the route the UAT plan names.
4. **Finish the one-time GitHub setup.** After the first run, protect `dev` and `main` so the 3 check jobs must pass before a merge. The images start out private on GHCR, so decide whether to make them public (docs/DEPLOYMENT.md, section 2).
5. **Remove the old local leftovers** (`checkin-test-db-setup` image, `modest_feynman` container). These are housekeeping only and don't affect the app.